BLOCKCHAIN SMART CONTRACT PRACTICAL + VIVA GUIDE

Practical 1: Customer Bank Account — Deposit, Withdraw, Show Balance

Purpose: A beginner-friendly walkthrough of the exact Remix workflow, the Solidity code, the mistakes encountered, and the core concepts needed for viva.

Environment used: Remix IDE • Remix VM (simulated blockchain) • Solidity 0.8.x

Keep this key distinction in mind: the Remix **Value** field is Ether sent along with a transaction; the **withdraw amount** is the separate function input telling the contract how much Ether to send back.

1. Practical statement and aim

Problem statement: Write a smart contract on a test network for a customer's bank account supporting deposit money, withdraw money, and show balance.

Aim: Write, compile, deploy, and interact with a Solidity smart contract that tracks each caller's Ether balance and supports deposit, withdrawal, and balance checking.

Context: This is a learning simulation, not a real bank. The contract keeps a balance record for each wallet address. Ether deposited is held by the contract, while the mapping records each customer's balance.

2. Final code (the version used)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {
    // Store each customer's balance by wallet address
    mapping(address => uint) balances;

    // Receive Ether and add it to the caller's recorded balance
    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // Withdraw Ether from the caller's recorded balance
    function withdraw(uint256 amount) public {
        require(
            balances[msg.sender] >= amount,
            "Insufficient balance"
        );

        // Update internal balance before sending Ether
        balances[msg.sender] -= amount;

        // Send Ether to the caller
        (bool success, ) =
            payable(msg.sender).call{value: amount}("");

        // Revert if the transfer did not succeed
        require(success, "Transfer failed");
    }

    // Read the caller's recorded balance
    function showBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

Note: `uint` is an alias for `uint256` in Solidity. This code uses a mapping keyed by wallet address, so each caller sees their own recorded balance.

3. Exact steps: from opening Remix to finishing

A. Open Remix and create the file

1. Open Remix IDE in your browser.
2. In the File Explorer, create a new Solidity file named **BankAccount.sol**.
3. Paste the code above and save it.

B. Compile

1. Click the Solidity Compiler icon on the left.
2. Choose a compiler compatible with `pragma solidity ^0.8.20;` (for example 0.8.20 or a later 0.8.x version).
3. Click **Compile BankAccount.sol**.

4. A successful compile means Solidity syntax and type checking passed. It does not yet mean the contract is deployed.

C. Select the test environment and deploy

1. Click **Deploy & Run Transactions**.
2. Choose **Remix VM** as Environment. It is a local simulated blockchain with test accounts and test Ether.
3. Select the **BankAccount** contract.
4. Set the deployment **Value** to **0**. The contract has no payable constructor, so do not send Ether during deployment.
5. Leave the **Gas Limit** on its default/automatic setting unless a specific gas error requires adjustment.
6. Click **Deploy**. The contract should appear under **Deployed Contracts**.

D. Deposit 1 Ether

1. Expand the deployed `BankAccount` contract.
2. In the transaction **Value** field, enter **1** and select **Ether** as the unit.
3. Click the orange **deposit** button (or Transact).
4. Wait for the green tick in the transaction log. It indicates the transaction succeeded.

Result: the caller's mapping balance increases by 1 ETH. 1 ETH = 1,000,000,000,000,000,000 wei (10^{18} wei).

E. Show the balance

1. Click the blue **showBalance** button (often labelled **Call**).
2. Remix displays the returned value. It may show a long hexadecimal value rather than decimal.

Expected after depositing 1 ETH: `1000000000000000000` wei, or `0xde0b6b3a7640000` in hexadecimal. This is 1 ETH.

F. Withdraw 0.5 Ether — the key input distinction

1. In the **withdraw(uint256)** input box, enter **500000000000000000**. This is the function argument: the amount to withdraw, in wei.
2. Separately, set the Remix transaction **Value** field to **0** (unit can be Wei or Ether). Do not put the withdrawal amount in Value.
3. Click **Transact** beside withdraw.
4. Wait for the green tick. The contract sends 0.5 ETH to the caller.

G. Verify the remaining balance

Click **showBalance** again. Expected: **500000000000000000 wei** = 0.5 ETH. This confirms the deposit and withdrawal accounting.

4. What happened during the errors?

Error 1: Deployment reverted when Value was 1 ETH.
The contract has no constructor marked `payable`. Solidity's default constructor behavior is non-payable, so deployment cannot accept Ether. Fix: set deployment Value to 0 and deploy. Deposit Ether later using the payable `deposit()` function.

Error 2: Withdrawal failed when the withdraw input was 0.
The transaction did not request the intended 0.5 ETH. Enter the withdrawal amount in the `withdraw` function input, not in the Value field.

Error 3: Withdrawal reverted when Value was 21 ETH or 0.5 ETH.
The Value field was sending Ether *to* the contract along with the call. `withdraw()` is not payable, so sending Ether with that call causes a revert. Fix: set transaction Value to 0; keep the amount only in the withdraw argument.

Transaction reverted means the attempted state changes are rolled back. In this case the withdrawal did not take effect and the recorded balance remained unchanged.

5. Remix fields: what should I enter?

Action	Remix Value field	Function input	Expected result
Deploy	0	None	Contract deployed
Deposit 1 ETH	1 Ether	None	Balance increases by 1 ETH
Show balance	0	None	Reads current caller balance
Withdraw 0.5 ETH	0	500000000000000000 (wei)	Balance decreases by 0.5 ETH
Show balance again	0	None	Remaining balance = 0.5 ETH

Gas Limit: Leave it on Remix's default/automatic setting for this practical. Gas is computation; Value is Ether being sent. They are different fields.

6. Blockchain and smart-contract theory (beginner level)

Blockchain: A distributed ledger maintained by a network of computers. Transactions are grouped into blocks, and blocks are cryptographically linked. This makes past records difficult to alter without network consensus.

Smart contract: A program deployed to a blockchain. It has code and persistent storage, and users interact with it by sending transactions or making read-only calls. It executes according to its programmed rules; it is not necessarily an AI or a legal contract.

Ethereum: A blockchain that supports programmable smart contracts. Ethereum uses Ether (ETH) as its native asset and the Ethereum Virtual Machine (EVM) to execute contract bytecode.

Solidity: A statically typed, high-level language commonly used to write EVM smart contracts. Other smart-contract languages include Vyper (EVM-focused), Rust (used in ecosystems such as Solana), and Move (used in Move-based chains). The language depends on the blockchain.

Remix IDE: A browser-based development environment for writing, compiling, deploying, and testing Solidity contracts. **Remix VM** is a simulated local blockchain, not a public testnet. A public testnet is a separate blockchain network where test tokens are used.

Ether (ETH) and wei: ETH is Ethereum's native cryptocurrency. Wei is the smallest unit commonly used in Solidity. 1 ETH = 10^{18} wei. Solidity's `msg.value` and payable amounts are denominated in wei.

Wallet/address: A wallet manages keys and helps a user interact with blockchain. The address is a public identifier. `msg.sender` is the address that called the current function. It is not the user's name or password.

Transaction: A signed request that changes blockchain state, such as depositing or withdrawing. It is included and executed by the network and consumes gas. On Remix VM, transactions are simulated.

Gas: A measure of computational work. Gas limit is the maximum gas allowed for a transaction; gas used is the amount consumed. Gas fees compensate network validators on public Ethereum. Remix VM uses simulated accounts and does not charge real ETH.

Compile vs deploy: Compile checks the source code and produces bytecode/ABI. Deploy sends the contract-creation transaction and stores the contract on the selected network. After deployment, the contract has an address and can be interacted with.

ABI: Application Binary Interface. It describes how external tools encode function calls and decode results. Remix uses the ABI to display buttons and inputs for functions.

State variable and mapping: A state variable persists in contract storage. `mapping(address => uint) balances` associates each address with an unsigned integer balance. Each address has its own mapping entry, initially zero.

payable / non-payable: `payable` allows a function or constructor to receive Ether. A function without `payable` is non-payable and rejects calls that send Ether. `deposit()` is payable; `withdraw()` is non-payable because it sends Ether out but does not accept Ether with the call.

Constructor: A special function that runs once when a contract is deployed to initialize state. If no constructor is written, Solidity provides a default one. It is non-payable unless explicitly declared payable. This practical has no custom constructor.

view vs pure: `view` can read state but cannot modify it. `pure` cannot read or modify contract state. A `view` call made through Remix's Call button is read-only and typically does not create a transaction. `deposit` and `withdraw` modify state, so they are transactions.

require and revert: `require(condition, message)` checks a condition. If false, execution reverts and state changes are rolled back. `revert` explicitly aborts execution. This contract uses `require` to prevent overdrafts and detect failed transfers.

Security note: This is a teaching contract, not production banking software. Real contracts need audits, access-control decisions, safe accounting, and protection against reentrancy and other attacks. The code reduces one classic risk by updating the balance before making the external call (Checks-Effects-Interactions), but it is not a complete audited banking system.

7. Top 41 viva questions and short answers

Use these as spoken answers. Aim for 1–3 sentences each; expand only if the examiner asks.

Q1. What is blockchain?

Answer: A distributed digital ledger maintained by multiple network participants. Blocks are linked cryptographically, making recorded data difficult to alter.

Q2. What is a smart contract?

Answer: A program deployed on a blockchain that executes its coded rules when users or other contracts interact with it.

Q3. What is the aim of this practical?

Answer: To create a bank-account-like contract supporting Ether deposit, withdrawal, and balance display.

Q4. What is Solidity?

Answer: A high-level, statically typed language used to write smart contracts for Ethereum and EVM-compatible blockchains.

Q5. Which other languages are used for smart contracts?

Answer: Vyper for EVM contracts; Rust in ecosystems such as Solana; and Move in Move-based blockchain ecosystems. The choice depends on the chain.

Q6. What is Remix IDE?

Answer: A browser-based IDE used to write, compile, deploy, and test Solidity smart contracts.

Q7. What is Remix VM?

Answer: A simulated local blockchain inside Remix with test accounts and test Ether. It is not the same as deploying to a public testnet.

Q8. What is a testnet?

Answer: A blockchain network used for development and testing with test tokens rather than real-value cryptocurrency.

Q9. What is Ethereum?

Answer: A programmable blockchain that supports smart contracts and uses the EVM for execution.

Q10. What is Ether?

Answer: ETH is Ethereum's native cryptocurrency, used for value transfer and paying transaction fees on the public network.

Q11. What is wei?

Answer: Wei is a very small denomination of Ether. 1 ETH equals 10^18 wei.

Q12. What is a wallet address?

Answer: A public blockchain identifier associated with a wallet. It is used to send and receive assets and interact with contracts.

Q13. What is msg.sender?

Answer: The address that called the current function. Here it identifies whose balance is being updated or read.

Q14. What is msg.value?

Answer: The amount of Ether attached to the current payable transaction, expressed in wei.

Q15. What is payable?

Answer: A Solidity modifier that allows a function or constructor to receive Ether.

Q16. What is a non-payable function?

Answer: A function not marked payable; it rejects a call that sends Ether along with it.

Q17. Why is deposit() payable?

Answer: Because the customer sends Ether with the deposit call, so the function must be allowed to receive it.

Q18. Why is withdraw() not payable?

Answer: It sends Ether out to the caller and does not need to accept Ether with the call. The transaction Value should therefore be zero.

Q19. What is a constructor?

Answer: A special function executed once when a contract is deployed, generally used to initialize state. If omitted, Solidity uses a default constructor.

Q20. Why did deployment fail when Value was 1 ETH?

Answer: The contract had no payable constructor. The deployment transaction attempted to send Ether to a non-payable constructor; setting deployment Value to zero fixes it.

Q21. What is a state variable?

Answer: A variable declared at contract level whose value is stored persistently in the contract's storage.

Q22. What is mapping in Solidity?

Answer: A key-value data structure. Here it maps each wallet address to that customer's unsigned integer balance.

Q23. Why use mapping(address => uint) balances?

Answer: To maintain separate balances for different wallet addresses without using one shared balance for all customers.

Q24. What is uint / uint256?

Answer: An unsigned integer type that cannot represent negative numbers. `uint` is an alias for `uint256` in Solidity.

Q25. What is a function?

Answer: A reusable block of code that performs an operation. This contract has deposit, withdraw, and showBalance functions.

Q26. What does public mean?

Answer: The function or variable can be accessed externally and internally, subject to Solidity's rules.

Q27. What does view mean?

Answer: The function can read contract state but cannot modify it.

Q28. What is the difference between view and pure?

Answer: View may read state; pure may neither read nor modify contract state.

Q29. Why is showBalance() view?

Answer: It only reads the caller's balance and does not change stored data.

Q30. What is require()?

Answer: A condition check. If it fails, execution reverts and the supplied error message is shown.

Q31. Why use require in withdraw?

Answer: To prevent a customer from withdrawing more than their recorded balance.

Q32. What is a transaction?

Answer: A signed operation sent to a blockchain. State-changing contract calls are transactions and consume gas.

Q33. What is gas?

Answer: A measure of computation needed to execute blockchain operations. On public Ethereum, gas is paid for in ETH.

Q34. What is gas limit?

Answer: The maximum amount of gas a transaction is allowed to consume. For this Remix practical, leave it at the default/automatic value.

Q35. What is the Remix Value field?

Answer: The amount of Ether sent along with the transaction, measured in the selected unit. It is separate from a function's input parameters.

Q36. What Value should be used for deployment?

Answer: Zero, because this contract has no payable constructor.

Q37. What Value should be used for deposit of 1 ETH?

Answer: Set Value to 1 and choose Ether, then click deposit. The payable function receives that ETH.

Q38. What Value should be used for withdrawal?

Answer: Zero. Enter the amount to withdraw in the withdraw function input, in wei.

Q39. Why did withdraw revert when Value was 21 ETH or 0.5 ETH?

Answer: Because Ether was being sent along with a call to a non-payable withdraw function. Set transaction Value to zero.

Q40. What is a revert?

Answer: It aborts execution and rolls back state changes from the failed transaction. A failed withdrawal does not deduct the balance.

Q41. What is the expected balance after depositing 1 ETH and withdrawing 0.5 ETH?

Answer: 0.5 ETH, which is 500000000000000000 wei.

8. Last-minute practical checklist

Before the viva, make sure you can explain these without looking:

- Why deploy with Value = 0.
- Why deposit has `payable` and why its Value is 1 ETH for the sample deposit.
- Why withdraw's Value = 0 but its input amount is 0.5 ETH in wei.
- What `msg.sender`, `msg.value`, mapping, `require`, and `view` mean.
- Why 1 ETH is 10^18 wei.
- Difference between compile, deploy, transaction, and read-only call.
- Why a reverted transaction leaves the balance unchanged.

One-sentence practical summary: “I wrote a Solidity BankAccount contract using a mapping to track balances by address, deployed it in Remix VM, deposited Ether through a payable function, withdrew a specified wei amount after checking the balance, and verified the remaining balance using a view function.”

Official learning references

- Remix IDE: remix.ethereum.org
- Solidity documentation: docs.soliditylang.org
- Ethereum developer documentation: ethereum.org/developers